

Visual construction & analysis of BoMs

This document provides specifications for a visual tool to design and implement for visually constructing and and/or analyzing a Bill of Materials (BoM) in manufacturing environments.

Background

A BoM is a centralized, structured inventory of all the physical raw materials, sub-assemblies, intermediate components, sub-components, and quantities needed to manufacture a finished product.

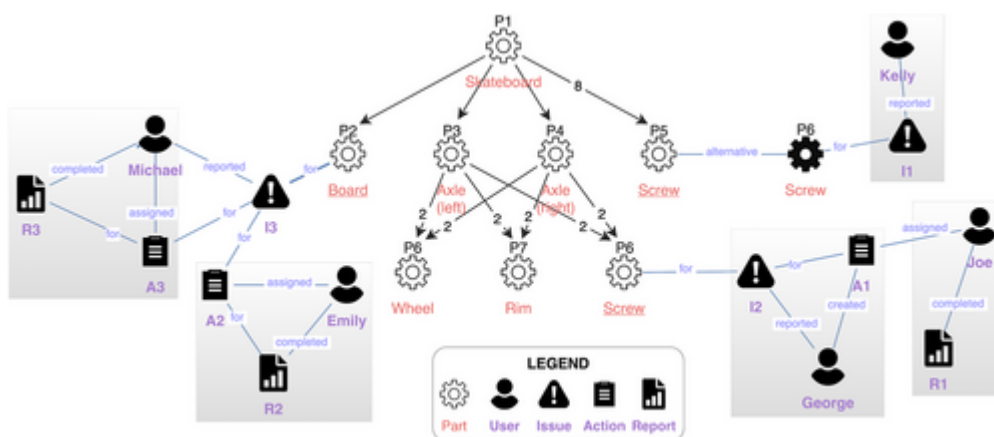
BoMs are mainly used in manufacturing for assembly sequence & production planning. They dictate the step-by-step build hierarchy (e.g., component → sub-assembly → final product) for assembly lines.

Proposed method

Traditional BoMs are managed as nested tables, multi-level indents, or tabular ERP lists, which obscure complex multi-level dependencies. A visual tool using hierarchical diagram notations directly addresses these limitations:

- **Intuitive Composition & Drag-and-Drop Authoring:** Allows engineers to visually nest sub-assemblies, link shared modules, and instantiate standardized component libraries without getting lost in deep tabular indentations.
- **Clear Multi-Level Lineage & Commonality Detection:** Exposes shared sub-assemblies across different product variants at a glance, highlighting opportunities for modular design, part standardization, and inventory consolidation.
- **Interactive Impact & Change Analysis:** Enables instant visual tracing of how modifying or replacing a single node (part revision, supply chain delay, material cost spike) propagates up to parent assemblies and the end product.
- **Visual Filtering & Compound Navigation:** Lets planners collapse/expand complex hierarchical sub-trees, color-code nodes by lead time, supplier risk, or cost thresholds, and quickly spot bottlenecks in deep multi-tier structures.

Visualizing a BoM also enables the integration of contextual metadata—such as logged issues, corrective actions, inspection reports, and resolution states—directly onto corresponding part nodes. Structurally, this models the product as a mixed graph: the core assembly hierarchy forms a directed acyclic graph (DAG), while associated issue tickets, action items, and resolution workflows attach as undirected satellite subgraphs around their respective components. Below is a simple example of such a mixed graph visualization taken from [1].



An initial prototype implementation exercise

In order to explore how visualization of such mixed graphs can be realized and how such graphs can be constructed and manipulated, we'd like to do an initial prototype as follows.

For this, we would like to use the [Cytoscape.js](#) library and its associated extensions [2] as our graph construction/rendering and manipulation library.

Graph model

In this implementation, we assume the following graph model (following the simple example above):

- Nodes: Part, User, Issue, Action, and Report
- Directed edges: Part -> Part (defines a whole-part relationship)
- Undirected edges:
 - Part - Issue (the issue is associated with the part)
 - Issue - User (the user reported the issue)
 - Issue - Action (the action was taken for the issue)
 - Issue - Report (the report was written for the issue)
 - User - Action (the user created the action to be taken)
 - User - Action (the action was assigned to the user)
 - User - Report (the user completed the report)

When visualizing these distinct node and edge types, we would like to use distinct icons for each node type and distinctly styled edges (perhaps with varying labels). Make sure that icons are freely licensed.

Graph construction

The user should be able to interactively create BoMs and related contextual metadata using the previously defined node and edge types. To illustrate the concept, each node and each edge type should have at least one property (input during construction)!

Inspecting graph entities

The user should be able to inspect any of the graph entities by selecting (left-clicking on them). Upon selection, the node/edge should be highlighted, and its properties should be shown on a right panel.

Graph layout

If the user would like to get help from an automated layout anytime during the visual construction and analysis, they might do so by using the HySE layout algorithm [3].

Complexity management

We should also implement some complexity management operations for managing the complexity of large BoMs. Such operations include (but are not limited to):

- Collapsing the downstream (all subparts) of a part
- Collapsing the upstream of a part (the ancestors)
- Expanding the up/downstream of a part (1 level, 3 levels, or all)

References

- [1] Dogrusoz et al. "[HySE: A Force-Directed Layout Algorithm for Directed and Mixed Graphs](#)", CGF, 2026
- [2] Cytoscape.js: [Graph theory \(network\) library for visualisation and analysis](#).
- [3] HySE: [A hybrid spring embedder layout library](#) on GitHub.